UNIVERSIDAD NACIONAL DE TRUJILLO

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



Fundamentos y Aplicación del Lenguaje Estructurado de Consultas (SQL) en la Gestión de Bases de Datos Relacionales

Autores:

Alcántara Molina Kenyi Isidro

Chávez Huayan, Tahiwo Oswmar

Díaz Vergara, Luis Favio

Gonzales Yarleque, Jagner Fabián

Pérez Cabanillas, Cristian Andree

Docente:

Ms. Ing. Zoraida Yanet Vidal Melgarejo

Experiencia Curricular:

Programación Orientada a Objetos II

Ciclo Académico: III

Fecha de entrega: 05 de junio

Guadalupe – 2026

Agradecimiento:

Queremos extender nuestro más profundo agradecimiento a nuestra docente, no solo por el conocimiento compartido, sino por su constante impulso para que avancemos en nuestra formación profesional. A nuestros compañeros, gracias por su genuina calidez, las risas y por crear un ambiente donde ha sido un placer aprender. Por último, nos gustaría hacer un reconocimiento especial a nuestro propio equipo. Gracias por la solidaridad inquebrantable a lo largo de este proyecto. Valoramos profundamente la empatía con la que supimos acompañarnos frente a las adversidades personales o económicas que surgieron de imprevisto. Ha sido un privilegio trabajar con un grupo tan unido, y esperamos seguir compartiendo nuevos retos juntos.

ÍNDICE:

RESUMEN v
ABSTRACT vi
MARCO TEÓRICO 9
Antecedentes: 9
Conceptos clave: 9
Modelos del proceso de desarrollo 10
Herramientas y estándares aplicables 11
Relación con la ingeniería de sistemas 11
METODOLOGÍA 12
DESARROLLO Y ANÁLISIS 14
CAPÍTULO I: FUNDAMENTOS Y DISEÑO ESTRUCTURAL (DDL) 14
1.1: Anatomía del Estándar SQL y Arquitectura Relacional 14
1.2: Modelado Lógico del Sistema de Gestión Académica 14
1.3: Traducción al Esquema Físico mediante DDL 15
1.4: Implementación y Ejecución de Estructuras 16
CAPÍTULO II: MANIPULACIÓN, SEGURIDAD Y CONTROL (DML Y DCL) 18
2.1: Gestión Transaccional y Propiedades ACID 18
2.2: Riesgos de Integración: La Inyección SQL (SQLi) 18
2.3: Mitigación de Vulnerabilidades mediante Consultas Parametrizadas 19
CAPÍTULO III: CONSULTAS AVANZADAS Y CASO PRÁCTICO 21
3.1: El Principio de Menor Privilegio en Bases de Datos 21
3.2: Gestión de Roles mediante Comandos DCL 21
3.3: Aplicación de Auditoría en el Caso Práctico 22
CAPÍTULO IV: ANÁLISIS DE DATOS Y CONSULTAS COMPLEJAS 23
4.1: Álgebra Relacional y Vinculación de Entidades (JOINs) 23
4.2: Procesamiento Escalar: Funciones de Agregado 23
4.3: Generación de Reportes Optimizados en el Caso Práctico 24
RESULTADOS 26
DISCUSIÓN 28
CONCLUSIONES Y RECOMENDACIONES 30
CONCLUSIONES 30
RECOMENDACIONES 31
REFERENCIAS 32

APÉNDICES 33
Apéndice A: Modelo Físico de la Base de Datos 33
Apéndice B: Diagramas Estructurales Del Sistema 34
Apéndice C: Script SQL del Caso Práctico 35
Apéndice D: Glosario De Términos Técnicos 38

RESUMEN

En el desarrollo de software moderno, la integración del modelo de la Programación Orientada a Objetos (POO) con el modelo relacional de las bases de datos suele presentar brechas técnicas críticas en los programadores, lo que degrada el rendimiento de las aplicaciones y las expone a vulnerabilidades graves como la inyección SQL (SQLi). Para abordar esta problemática, esta investigación aplicó un enfoque cualitativo, descriptivo y aplicado bajo el modelo de desarrollo en cascada, sustentado en una revisión bibliográfica sistemática de la literatura técnico-académica reciente. Como resultado, se diseñó e implementó con éxito en el motor MySQL 8 un esquema relacional completamente normalizado para un Sistema de Gestión Académica. La seguridad del sistema se garantizó mediante la mitigación de ataques informáticos utilizando consultas preparadas y parametrizadas para la inserción de datos, complementado con un control de accesos estricto basado en el principio del menor privilegio mediante instrucciones de control de datos. Asimismo, la eficiencia del sistema se optimizó al delegar la ejecución de consultas analíticas complejas directamente al motor de la base de datos. De este modo, se demuestra que el dominio avanzado de SQL puro constituye un pilar arquitectónico imprescindible en la Ingeniería de Sistemas, siendo un requisito innegociable para asegurar la persistencia, la seguridad, la escalabilidad y la correcta traducción de las reglas de negocio en aplicaciones empresariales fiables.

Palabras Clave: SQL, Base de datos relacional, Inyección SQL, Seguridad de datos, Modelo en cascada.

ABSTRACT

In modern software development, the integration of the Object-Oriented Programming (OOP) model with the relational database model often presents critical technical gaps among programmers, which degrades application performance and exposes them to severe vulnerabilities such as SQL injection (SQLi). To address this issue, this research applied a qualitative, descriptive, and applied approach under the waterfall development model, supported by a systematic bibliographic review of recent technical-academic literature. As a result, a fully normalized relational schema for an Academic Management System was successfully designed and implemented in the MySQL 8 engine. System security was ensured by mitigating cyberattacks using prepared and parameterized queries for data insertion, complemented by strict access control based on the principle of least privilege through data control instructions. Furthermore, system efficiency was optimized by delegating the execution of complex analytical queries directly to the database engine. Thus, it is demonstrated that advanced mastery of pure SQL constitutes an essential architectural pillar in Systems Engineering, being a non-negotiable requirement to ensure persistence, security, scalability, and the correct translation of business rules in reliable enterprise applications.

Keywords: SQL, Relational database, SQL injection, Data security, Waterfall mode

FUNDAMENTOS Y APLICACIÓN DEL LENGUAJE ESTRUCTURADO DE CONSULTAS (SQL) EN LA GESTIÓN DE BASES DE DATOS RELACIONALES

El desarrollo de software moderno exige gestionar grandes volúmenes de información de forma segura, estructurada y eficiente. Dentro de la Ingeniería de Sistemas, las bases de datos relacionales representan la infraestructura principal para lograr este propósito, y el Lenguaje Estructurado de Consultas (SQL) actúa como la herramienta estándar universal para interactuar con ellas. Entender cómo se modelan y manipulan los datos a través de este lenguaje no es simplemente un requisito de almacenamiento, sino la base arquitectónica obligatoria para construir sistemas robustos y escalables a nivel empresarial (Silberschatz, Korth & Sudarshan, 2020).

Sin embargo, en el desarrollo práctico de software existe una brecha técnica recurrente cuando se intenta integrar el paradigma de Programación Orientada a Objetos (POO) con el modelo relacional. Muchos programadores logran dominar la creación de clases y la instanciación de objetos, pero fallan al momento de persistir y consultar esa información mediante código SQL optimizado. Esta desconexión no solo genera graves cuellos de botella en el rendimiento de las aplicaciones por consultas mal estructuradas, sino que abre la puerta a fallos críticos de seguridad. Si los desarrolladores no aplican buenas prácticas desde la sintaxis de la consulta, los sistemas quedan expuestos a vulnerabilidades severas, como los ataques de inyección SQL (Mendoza & Pérez, 2023).

Por este motivo, el presente informe se justifica en la necesidad ineludible de dominar SQL como un pilar fundamental para cualquier ingeniero de sistemas. Conocer a profundidad cómo construir esquemas, manipular registros y establecer permisos de usuario permite diseñar aplicaciones que sean rápidas, consistentes y seguras. A nivel académico, consolidar esta base técnica es vital dentro de la experiencia curricular, ya que proporciona las herramientas necesarias para que el diseño lógico de las clases en Java (o cualquier lenguaje orientado a objetos) tenga un respaldo eficiente en la base de datos.

Para abordar esta temática con el rigor técnico necesario, se ha establecido como objetivo general: Analizar la estructura teórica, los comandos principales y la aplicación práctica del estándar SQL para su correcta implementación en sistemas de información. Para garantizar el cumplimiento de esta meta, se han definido los siguientes cinco objetivos específicos:

1. Describir los fundamentos teóricos y la arquitectura general del estándar SQL dentro de los sistemas de bases de datos relacionales.

2. Explicar la sintaxis y utilidad práctica de los comandos de Definición de Datos (DDL) en la creación y alteración de esquemas.
3. Detallar la implementación de los comandos de Manipulación (DML) y Control de Datos (DCL) para la gestión segura de registros y permisos de acceso.
4. Demostrar el uso estructurado de cláusulas, operadores lógicos y funciones de agregado en la resolución de consultas con múltiples condiciones.
5. Desarrollar un caso práctico mediante la escritura de scripts SQL para validar empíricamente la teoría en un entorno de base de datos relacional.

El alcance de este trabajo se limita estrictamente al estudio de la sintaxis estándar de SQL, abarcando desde el modelado de tablas básicas hasta la ejecución de consultas con agrupación y agregación de datos. Como limitación, el documento no explorará la programación de bases de datos avanzada (como la creación de *Stored Procedures* o *Triggers*) ni lenguajes procedimentales específicos como PL/SQL, ya que el propósito es asentar las bases universales del lenguaje sin interferir con temas posteriores del curso.

En cuanto a su organización, el informe inicia con un marco teórico que consolida los conceptos clave apoyados en literatura académica. A continuación, se expone la metodología descriptiva que guió el análisis, para dar paso a la sección de desarrollo, donde se evalúan las instrucciones SQL y se integran con un modelo práctico. Finalmente, se discuten los hallazgos en relación con la teoría, se presentan las conclusiones del equipo y se anexan los diagramas y códigos desarrollados.

MARCO TEÓRICO

Antecedentes:

La literatura científica demuestra que estructurar, optimizar y asegurar las consultas relacionales es crítico para el rendimiento y la estabilidad de los sistemas de información:

- **Optimización y Rendimiento:** García y López (2022), en *Optimización de consultas SQL en sistemas de información académica*, analizaron el impacto del diseño de índices y la reestructuración de sentencias DML en entornos de alta concurrencia. Demostraron que una correcta indexación y la reducción de subconsultas anidadas mitigan los cuellos de botella y optimizan la eficiencia operativa.
- **Arquitectura y Seguridad:** Mendoza y Pérez (2023), en *Prevención de vulnerabilidades en aplicaciones orientadas a objetos mediante buenas prácticas en SQL*, examinaron los riesgos de interacción entre la aplicación y el motor relacional. Advirtieron que las consultas dinámicas exponen el sistema a inyecciones de código (SQLi), concluyendo que la implementación estricta de consultas preparadas (*parameterized queries*) y capas de abstracción robustas es indispensable para garantizar la integridad y confidencialidad de la información.

Conceptos clave:

Lenguaje Estructurado de Consultas (SQL): Es el lenguaje declarativo estándar diseñado universalmente para la creación, manipulación y control de bases de datos relacionales. A diferencia de los lenguajes de programación imperativos, SQL se enfoca en definir "qué" datos se necesitan recuperar o modificar, delegando al motor de la base de datos la optimización de "cómo" ejecutar dicha tarea (Smith & Johnson, 2024).

Lenguaje de Definición de Datos (DDL): Comprende el subconjunto de comandos de SQL orientados a definir y modificar las estructuras lógicas y físicas que almacenan la información, tales como bases de datos, tablas, índices y vistas. Las instrucciones principales incluyen *CREATE, ALTER* y DROP, las cuales actúan directamente sobre el esquema relacional y la metadata del sistema (Silberschatz, Korth & Sudarshan, 2020).

Lenguaje de Manipulación de Datos (DML): Es el conjunto de instrucciones utilizadas para gestionar los registros individuales dentro de las tablas previamente definidas. Incluye las operaciones fundamentales de inserción *(INSERT*), actualización (*UPDATE*), eliminación (*DELETE*) y recuperación de información (*SELECT*), siendo esta última la herramienta más

utilizada para la interacción entre aplicaciones de software y la capa de persistencia (Beaulieu, 2020).

Lenguaje de Control de Datos (DCL): Abarca los comandos encargados de gestionar la seguridad, las autorizaciones y el acceso concurrente a la base de datos. Mediante instrucciones como *GRANT* (otorgar privilegios) y *REVOKE* (revocar privilegios), los administradores aseguran que solo los usuarios o aplicaciones con las credenciales adecuadas puedan ejecutar operaciones DDL o DML (Coronel & Morris, 2023).

Cláusulas y Funciones de Agregado: Las cláusulas (como *WHERE, GROUP BY, ORDER BY*) son condiciones sintácticas que filtran y organizan el conjunto de resultados de una consulta. Por su parte, las funciones de agregado (*SUM, AVG, COUNT, MAX, MIN*) son operaciones matemáticas que procesan múltiples filas de una columna para devolver un único valor escalar, siendo esenciales para la generación de reportes y análisis de datos masivos (Oracle, 2025).

Modelos del proceso de desarrollo

En el ámbito de la ingeniería de software, el ciclo de vida de un sistema puede abordarse mediante diversas metodologías. Cuando un proyecto demanda predictibilidad, un control riguroso y una definición exhaustiva de requisitos desde las fases iniciales, se opta por el enfoque tradicional o modelo en cascada (Waterfall).

Este modelo se organiza en una secuencia lineal de etapas consecutivas, donde el inicio de una fase depende de la culminación de la anterior:

1. **Análisis de Requisitos:** Determinación precisa de las necesidades del negocio y especificación de los datos que se deben procesar y almacenar.
2. **Diseño de la Arquitectura y Base de Datos:** Traducción de las necesidades a esquemas lógicos y físicos formalizados.
3. **Implementación:** Codificación del software y construcción física de las tablas, restricciones de integridad e índices en el gestor de bases de datos.
4. **Pruebas e Integración:** Validación del correcto funcionamiento del sistema y verificación de la consistencia de los datos bajo escenarios de carga.
5. **Mantenimiento:** Aplicación de optimizaciones, parches de seguridad y ajustes post-despliegue.

Herramientas y estándares aplicables

La gestión relacional de datos se rige por especificaciones técnicas que garantizan su robustez e interoperabilidad:

- **Estándar ISO/IEC 9075:** La norma ISO/IEC 9075-1:2023 define el marco conceptual y sintáctico de SQL, asegurando consistencia y portabilidad multiplataforma.
- **Modelo Relacional y Normalización:** La información se organiza en tablas estructuradas en filas (tuplas) y columnas (atributos) (Elmasri y Navathe, 2021; Coronel y Morris, 2023). Las Formas Normales (Date, 2019) evitan redundancias y anomalías de inserción, actualización o borrado.
- **SQL:** Herramienta estándar para definir (DDL), manipular (DML) y controlar (DCL) datos en un RDBMS, permitiendo desde selecciones simples hasta *joins* y agrupamientos (Beaulieu, 2020).
- **Implementación y Motores:** En producción, el uso de manuales técnicos como el *MySQL 8.4 Reference Manual* (Oracle, 2025) es vital para adaptar la sintaxis, optimizar el rendimiento, configurar el almacenamiento y garantizar las propiedades transaccionales ACID (Aislamiento, Consistencia, Integridad y Durabilidad).

Relación con la ingeniería de sistemas

La gestión de bases de datos relacionales es un pilar nuclear dentro de la Ingeniería de Sistemas, disciplina que se encarga del diseño, integración y optimización de sistemas complejos. La relación directa se manifiesta a través de los siguientes puntos:

Integración de Componentes: La base de datos actúa como la capa de persistencia central que unifica la lógica de negocio del servidor, las interfaces de usuario y las comunicaciones de red, asegurando la cohesión del sistema global.

Garantía de Atributos de Calidad: Un diseño metodológico estricto permite que el sistema cumpla con métricas de alta disponibilidad, seguridad informática y escalabilidad, soportando el incremento transaccional sin comprometer los tiempos de respuesta.

Modelamiento de Procesos Organizacionales: La capacidad de traducir reglas de negocio del mundo real a estructuras abstractas de datos es fundamental para automatizar flujos de trabajo, centralizar la información operativa y facilitar la toma de decisiones basada en datos fiables.

METODOLOGÍA

Para la elaboración del presente informe y el diseño del caso práctico, se estructuró una metodología alineada con los estándares de la ingeniería de software. Según su propósito, el estudio se clasifica como una investigación de tipo **descriptiva y aplicada**. Es descriptiva debido a que detalla la arquitectura, las normativas (como el estándar ISO/IEC 9075) y la sintaxis estructural del lenguaje SQL abordadas en el marco teórico. Simultáneamente, adquiere un carácter aplicado dado que los fundamentos de modelado relacional y optimización se trasladan a la resolución de un problema técnico concreto, mediante la codificación de un sistema de persistencia de datos.

El enfoque adoptado es de naturaleza **cualitativa**. El análisis no se centró en la recolección de variables numéricas ni muestreos probabilísticos, sino en la evaluación lógica de la arquitectura de datos, la revisión de literatura especializada y la validación estructural de sentencias SQL. Este enfoque resulta ser el más adecuado y justificado para el tema, ya que en el diseño de bases de datos relacionales, la coherencia de las dependencias funcionales y la mitigación de vulnerabilidades priman sobre el análisis cuantitativo.

En cuanto a las técnicas de recolección de información, se empleó la **revisión bibliográfica sistemática** como instrumento principal. Los criterios de selección de fuentes exigieron el uso exclusivo de artículos científicos arbitrados, textos académicos sobre fundamentos de bases de datos y manuales técnicos oficiales (como la documentación de MySQL 8.4). Se priorizó la literatura publicada en los últimos cinco años para asegurar la vigencia de las técnicas de seguridad, descartando de manera estricta fuentes colaborativas sin revisión por pares.

Finalmente, el procedimiento de análisis para la demostración práctica se ejecutó siguiendo las directrices del **modelo tradicional en cascada (Waterfall)** expuesto en el marco conceptual, dividiendo el trabajo en las siguientes fases secuenciales:

1. **Análisis y Diseño de la Arquitectura:** Se definieron los requisitos de información del caso de estudio y se construyó el modelo lógico. En esta etapa se aplicaron las reglas de normalización para evitar anomalías de actualización y redundancias en los registros.
2. **Implementación:** Se procedió con la codificación física en el motor relacional. Se emplearon comandos DDL para la creación de esquemas, tablas y restricciones de integridad, y comandos DCL para garantizar un entorno de acceso controlado.

3. **Pruebas de Integración:** Cabe destacar que, para la presentación del análisis en el presente documento, se adoptó un **enfoque de aplicación transversal**. En lugar de segregar la teoría de la práctica, el caso de estudio del Sistema de Gestión Académica actúa como hilo conductor a lo largo de todos los capítulos, permitiendo evidenciar la ejecución de los comandos SQL a la par de su fundamentación teórica. Adicionalmente, para preservar la densidad académica y la fluidez de lectura en los capítulos principales, la totalidad del código fuente ejecutable y el modelado integral de la base de datos se han consolidado en la sección de Apéndices, garantizando la replicabilidad técnica del proyecto durante la sustentación.

DESARROLLO Y ANÁLISIS

CAPÍTULO I: FUNDAMENTOS Y DISEÑO ESTRUCTURAL (DDL)

1.1: Anatomía del Estándar SQL y Arquitectura Relacional

Dentro del ecosistema del desarrollo de software, la gestión de la persistencia de datos requiere un enfoque semántico distinto al utilizado en la programación lógica. Mientras que la Programación Orientada a Objetos (POO) empleado en lenguajes como Java es de naturaleza imperativa —donde el desarrollador debe instruir al procesador paso a paso sobre cómo resolver un algoritmo—, el Lenguaje Estructurado de Consultas (SQL) opera bajo un modelo netamente declarativo. Esta característica arquitectónica implica que el ingeniero define "qué" información desea estructurar o recuperar, delegando al optimizador del motor relacional la responsabilidad de trazar el plan de ejecución más eficiente para acceder a los bloques de memoria física.

Para que esta comunicación entre la capa de aplicación y la base de datos sea cohesiva, independiente del proveedor y altamente portable, la sintaxis de SQL no es arbitraria, sino que está rigurosamente gobernada por especificaciones internacionales. El estándar ISO/IEC 9075-1:2023 define el marco conceptual y las reglas sintácticas fundamentales que todos los sistemas de gestión de bases de datos relacionales (RDBMS) deben implementar para garantizar la interoperabilidad. Gracias a esta estandarización, las estructuras de datos diseñadas en la fase de análisis pueden migrar entre motores (como MySQL, PostgreSQL o SQL Server) con modificaciones mínimas, asegurando la escalabilidad a largo plazo del sistema.

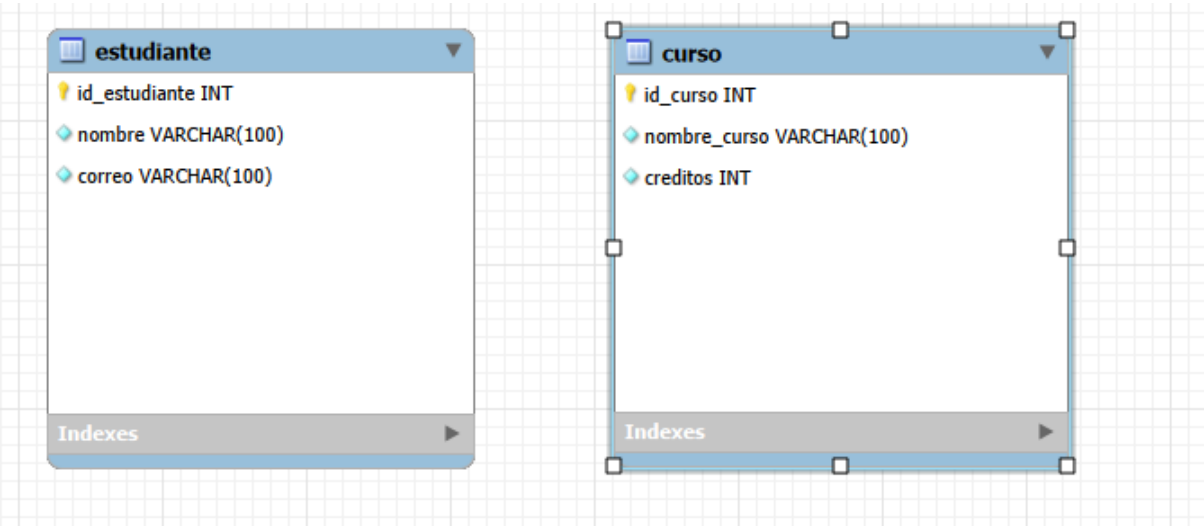
1.2: Modelado Lógico del Sistema de Gestión Académica

La transición desde los requerimientos abstractos del mundo real hacia un esquema de base de datos funcional exige un modelado previo. Para fines de este análisis y como caso de estudio demostrativo, se ha diseñado un módulo central de un Sistema de Gestión Académica, enfocado en el proceso de matrícula universitaria. En el contexto de la POO, este escenario se representaría típicamente mediante dos clases principales interconectadas: Estudiante y Curso.

Al trasladar esta lógica al modelo relacional, estas clases se transforman en tablas físicas, y sus atributos (identificadores, nombres, créditos) se convierten en las columnas que

albergarán los registros. El diseño de este esquema no es empírico; debe obedecer a las reglas de normalización para asegurar la atomicidad de los datos y evitar dependencias transitivas que generen anomalías al momento de actualizar o eliminar registros. En la siguiente figura se detalla el modelado físico planteado para el caso de estudio, estableciendo los tipos de datos y la clave principal para cada entidad.

Figura 1: Modelo físico de las tablas correspondientes al módulo de gestión académica



Nota. Diseño estructurado en el entorno de MySQL Workbench. Elaboración propia.

1.3: Traducción al Esquema Físico mediante DDL

Una vez validado el modelo, la estructuración en el servidor se ejecuta mediante el Lenguaje de Definición de Datos (DDL). Este subconjunto crítico de SQL es el encargado directo de interactuar con el diccionario de datos del gestor relacional, permitiendo construir, modificar o destruir la arquitectura donde residirá la información. A diferencia de otros comandos, las instrucciones DDL generan cambios estructurales irreversibles y automáticos en el esquema (autocommit), por lo que su ejecución en entornos de producción exige planificación y privilegios administrativos.

Los comandos fundamentales que componen este sublenguaje permiten abarcar todo el ciclo de vida de una tabla. La instrucción CREATE se utiliza para instanciar nuevas entidades y definir sus restricciones de integridad (llaves primarias y foráneas). Si los requerimientos del negocio cambian con el tiempo, la instrucción ALTER permite modificar la estructura de una tabla existente (por ejemplo, agregando nuevas columnas) sin comprometer la información ya almacenada. Finalmente, la instrucción DROP se emplea para erradicar por completo un

objeto del esquema, liberando el espacio en el almacenamiento físico. A continuación, se presenta una tabla comparativa que sintetiza el impacto de estos comandos.

Tabla 1: Análisis funcional de las principales sentencias del sublenguaje DDL

Comando	Función Arquitectónica	Impacto en el Motor de Base de Datos
CREATE	Construcción inicial.	Reserva espacio en memoria y registra los metadatos de la nueva estructura en el catálogo del sistema.
ALTER	Evolución del esquema.	Modifica la firma de la tabla (tipos de datos, adición/eliminación de columnas o *constraints*) en tiempo de ejecución.
DROP	Destrucción de objetos.	Elimina de manera irreversible la tabla, sus índices asociados y todos los registros contenidos.
TRUNCATE	Reinicio estructural.	Vacia la tabla eliminando las tuplas masivamente, pero preserva la estructura y metadatos intactos para futuras inserciones.

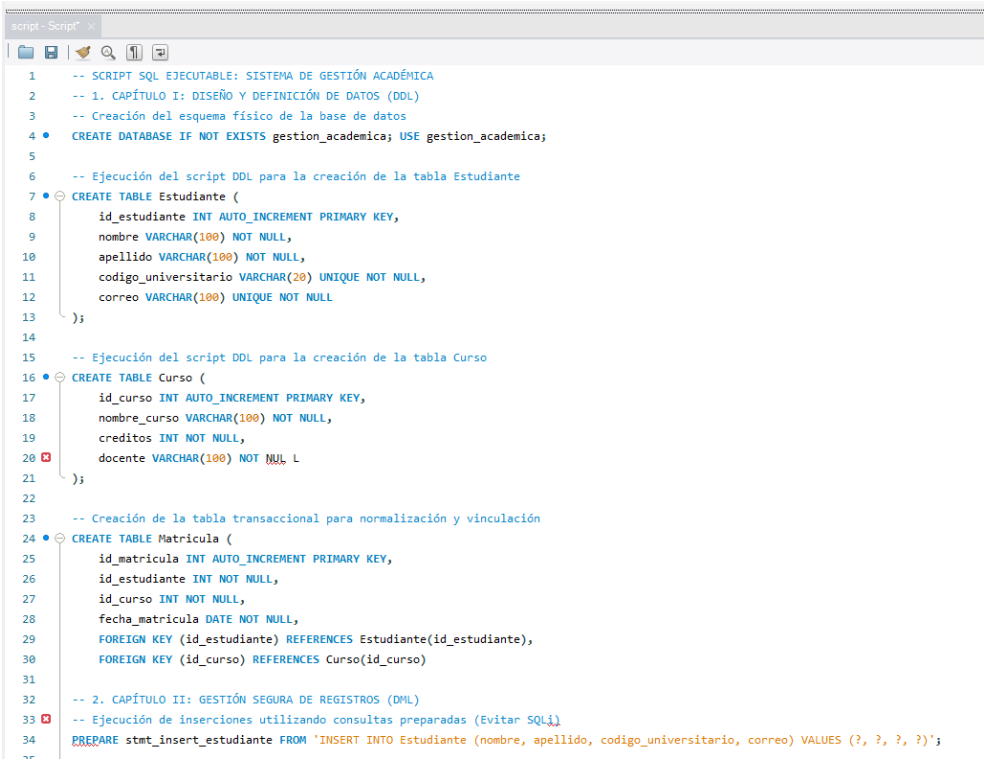
Nota. Análisis derivado de los fundamentos de diseño e implementación de bases de datos relacionales.

1.4: Implementación y Ejecución de Estructuras

La teoría del diseño relacional converge en la ejecución práctica del código SQL. Para materializar el caso de estudio del Sistema de Gestión Académica, se codificaron los scripts DDL correspondientes en el motor MySQL versión 8. Como se observa en la evidencia técnica adjunta, se utilizó la sentencia `CREATE TABLE` definiendo explícitamente dominios variados,

tales como INT para valores numéricos, VARCHAR para cadenas de texto como apellidos o correos, y tipos temporales como DATE para registrar la fecha exacta de las matrículas. Asimismo, se garantizó la integridad de entidad mediante la cláusula PRIMARY KEY y restricciones UNIQUE para evitar la duplicidad de códigos universitarios.

Figura 2: Ejecución exitosa de sentencias DDL para la instanciación de las entidades del sistema.



```sql
-- SCRIPT SQL EJECUTABLE: SISTEMA DE GESTIÓN ACADÉMICA
-- 1. CAPÍTULO I: DISEÑO Y DEFINICIÓN DE DATOS (DDL)
-- Creación del esquema físico de la base de datos
CREATE DATABASE IF NOT EXISTS gestion_academica; USE gestion_academica;

-- Ejecución del script DDL para la creación de la tabla Estudiante
CREATE TABLE Estudiante (
    id_estudiante INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    codigo_universitario VARCHAR(20) UNIQUE NOT NULL,
    correo VARCHAR(100) UNIQUE NOT NULL
);

-- Ejecución del script DDL para la creación de la tabla Curso
CREATE TABLE Curso (
    id_curso INT AUTO_INCREMENT PRIMARY KEY,
    nombre_curso VARCHAR(100) NOT NULL,
    creditos INT NOT NULL,
    docente VARCHAR(100) NOT NUL L
);

-- Creación de la tabla transaccional para normalización y vinculación
CREATE TABLE Matricula (
    id_matricula INT AUTO_INCREMENT PRIMARY KEY,
    id_estudiante INT NOT NULL,
    id_curso INT NOT NULL,
    fecha_matricula DATE NOT NULL,
    FOREIGN KEY (id_estudiante) REFERENCES Estudiante(id_estudiante),
    FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)

-- 2. CAPÍTULO II: GESTIÓN SEGURA DE REGISTROS (DML)
-- Ejecución de inserciones utilizando consultas preparadas (Evitar SQLi)
PREPARE stmt_insert_estudiante FROM 'INSERT INTO Estudiante (nombre, apellido, codigo_universitario, correo) VALUES (?, ?, ?, ?)';
```

Nota. Script de creación ejecutado en el gestor MySQL evidenciando la sintaxis declarativa. Elaboración propia.

Para integrar lo tratado en esta sección, señalamos como grupo que, el diseño y la implementación del esquema físico mediante el sublenguaje DDL no es una simple tarea de almacenamiento, sino la fase más crítica del desarrollo de software. Un modelado relacional correcto, regido por el estándar ISO y las normas de integridad estructural, garantiza que el puente entre las clases del paradigma orientado a objetos y la base de datos sea sólido, previendo redundancias y asegurando la escalabilidad del sistema desde su etapa fundacional.

CAPÍTULO II: MANIPULACIÓN, SEGURIDAD Y CONTROL (DML Y DCL)

2.1: Gestión Transaccional y Propiedades ACID

Una vez que la arquitectura relacional ha sido instanciada físicamente mediante el Lenguaje de Definición de Datos (DDL), las estructuras resultantes actúan como contenedores vacíos. La utilidad real de un sistema de bases de datos radica en su capacidad para recibir, actualizar y distribuir información en tiempo real. Esta interacción dinámica se ejecuta exclusivamente a través del Lenguaje de Manipulación de Datos (DML), el cual abarca las instrucciones INSERT, UPDATE, DELETE y SELECT.

Sin embargo, en el contexto de la Ingeniería de Sistemas, la ejecución de estos comandos no se trata de una simple escritura en disco. Las operaciones DML en motores robustos como MySQL operan bajo el modelo transaccional, lo que exige el cumplimiento estricto de las propiedades ACID: Aislamiento, Consistencia, Integridad y Durabilidad. Esto garantiza que si una aplicación orientada a objetos envía una instrucción masiva de actualización de notas para la entidad Curso, la transacción se completará en su totalidad o se revertirá (rollback) si ocurre un fallo en el servidor, evitando que la base de datos quede en un estado corrupto o inconsistente.

2.2: Riesgos de Integración: La Inyección SQL (SQLi)

La integración entre el paradigma de Programación Orientada a Objetos (POO) y el modelo relacional presenta desafíos críticos en la capa de persistencia. Típicamente, las aplicaciones de software capturan datos a través de interfaces de usuario y los envían al motor de base de datos para ser almacenados. El error arquitectónico más común, y a la vez más letal, ocurre cuando los desarrolladores construyen sentencias DML concatenando variables de texto dinámicas directamente en el código fuente de la aplicación.

Esta mala práctica expone inmediatamente al sistema a vulnerabilidades de Inyección SQL (SQLi). Al concatenar cadenas sin sanitización, un atacante informático puede introducir caracteres de escape (como comillas simples o guiones) dentro de un formulario web, alterando la estructura semántica original de la consulta. Como resultado, el motor relacional no distingue entre la instrucción legítima del programador y el código malicioso inyectado, procediendo a ejecutar comandos no autorizados que pueden ir desde la elusión de validaciones de acceso (bypass de autenticación) hasta el borrado masivo de tablas críticas mediante el uso indebido de operadores lógicos condicionales. El análisis de vulnerabilidades en aplicaciones modernas demuestra que la seguridad defensiva debe implementarse obligatoriamente desde el diseño de la consulta.

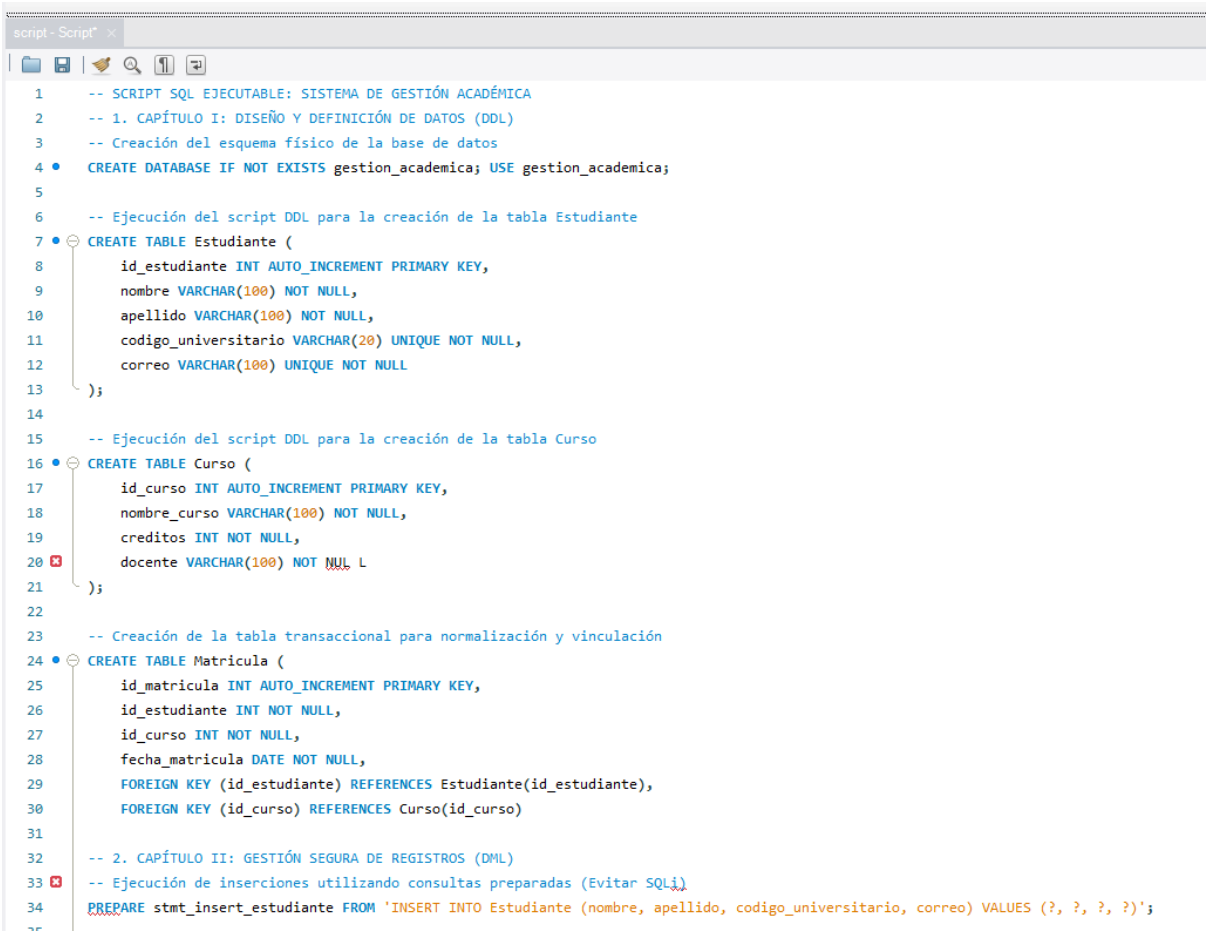
2.3: Mitigación de Vulnerabilidades mediante Consultas Parametrizadas

Para contrarrestar la amenaza de inyecciones de código y garantizar la integridad de la información en el Sistema de Gestión Académica, la literatura técnica exige el abandono de las consultas dinámicas en favor del uso estricto de consultas preparadas (*parameterized queries*). Esta técnica metodológica separa de manera absoluta la lógica de programación del contenido de los datos (payload).

El funcionamiento de este mecanismo consta de dos fases. En la primera fase, la aplicación envía al motor de base de datos la plantilla de la consulta DML (por ejemplo, una instrucción INSERT), utilizando marcadores de posición (?) en lugar de los valores reales. El gestor relacional compila, optimiza y bloquea la estructura lógica de la sentencia. En la segunda fase, la aplicación transmite únicamente los datos escalares proporcionados por el usuario. Debido a que la consulta ya ha sido precompilada, el motor trata estos valores estrictamente como texto literal y nunca como instrucciones ejecutables, neutralizando matemáticamente cualquier intento de alterar la sintaxis.

Para validar este marco teórico, se procedió a poblar las entidades lógicas del sistema académico. El siguiente script evidencia cómo se estructura una inserción segura para la tabla `Estudiante`, pasando múltiples parámetros (nombre, apellido, código universitario y correo) a través de variables de sesión preparadas y la preparación de sentencias en el entorno del motor de base de datos.

Figura 3: Inserción de registros mediante consultas parametrizadas para la mitigación de inyecciones SQL



```sql
-- SCRIPT SQL EJECUTABLE: SISTEMA DE GESTIÓN ACADÉMICA
-- 1. CAPÍTULO I: DISEÑO Y DEFINICIÓN DE DATOS (DDL)
-- Creación del esquema físico de la base de datos
CREATE DATABASE IF NOT EXISTS gestion_academica; USE gestion_academica;

-- Ejecución del script DDL para la creación de la tabla Estudiante
CREATE TABLE Estudiante (
    id_estudiante INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    codigo_universitario VARCHAR(20) UNIQUE NOT NULL,
    correo VARCHAR(100) UNIQUE NOT NULL
);

-- Ejecución del script DDL para la creación de la tabla Curso
CREATE TABLE Curso (
    id_curso INT AUTO_INCREMENT PRIMARY KEY,
    nombre_curso VARCHAR(100) NOT NULL,
    creditos INT NOT NULL,
    docente VARCHAR(100) NOT NUL L
);

-- Creación de la tabla transaccional para normalización y vinculación
CREATE TABLE Matricula (
    id_matricula INT AUTO_INCREMENT PRIMARY KEY,
    id_estudiante INT NOT NULL,
    id_curso INT NOT NULL,
    fecha_matricula DATE NOT NULL,
    FOREIGN KEY (id_estudiante) REFERENCES Estudiante(id_estudiante),
    FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)

-- 2. CAPÍTULO II: GESTIÓN SEGURA DE REGISTROS (DML)
-- Ejecución de inserciones utilizando consultas preparadas (Evitar SQLi)
PREPARE stmt_insert_estudiante FROM 'INSERT INTO Estudiante (nombre, apellido, codigo_universitario, correo) VALUES (?, ?, ?, ?)';
```

Nota. Script de validación técnica ejecutado para garantizar la persistencia segura de la información. Elaboración propia.

De este capítulo englobamos que, la manipulación de datos en un entorno empresarial no se limita a la correcta escritura sintáctica de las instrucciones DML; implica una responsabilidad crítica orientada a la seguridad informática. La implementación sistemática de consultas parametrizadas representa el estándar de la industria para erradicar las vulnerabilidades de inyección SQL, asegurando que la integración entre las clases de la aplicación y las tablas relacionales sea estructuralmente impermeable a manipulaciones maliciosas.

CAPÍTULO III: CONSULTAS AVANZADAS Y CASO PRÁCTICO

3.1: El Principio de Menor Privilegio en Bases de Datos

En el diseño arquitectónico de sistemas de información contemporáneos, la seguridad no se delega de manera exclusiva a la aplicación cliente o al servidor web; debe implementarse bajo una estrategia de "defensa en profundidad" donde la capa de persistencia actúa como la última y más importante línea de contención. Uno de los antipatrones de diseño más críticos y frecuentes en el desarrollo de software universitario y empresarial es la conexión de la aplicación (por ejemplo, una API desarrollada en Java) hacia la base de datos utilizando las credenciales del usuario raíz (conocido como `root` o `sa`).

Este enfoque viola inherentemente el Principio de Menor Privilegio (PoLP, por sus siglas en inglés), el cual dictamina que un proceso, usuario o programa debe poseer únicamente los permisos estrictamente necesarios para ejecutar su función designada, y nada más. Si una aplicación que solo necesita leer reportes académicos se conecta como administrador global, un eventual fallo de seguridad (como la inyección SQL analizada en el capítulo anterior) le otorgaría al atacante el poder de destruir esquemas completos. Para mitigar este riesgo estructural, los motores relacionales implementan un sublenguaje dedicado exclusivamente a la segregación de funciones operativas.

3.2: Gestión de Roles mediante Comandos DCL

El Lenguaje de Control de Datos (DCL) proporciona la semántica necesaria para gobernar el acceso concurrente y la autorización dentro del ecosistema relacional. Este subconjunto se articula principalmente a través de dos sentencias declarativas: `GRANT`, utilizada para otorgar privilegios específicos sobre objetos determinados, y `REVOKE`, diseñada para retirar dichas concesiones (Coronel & Morris, 2023). La granularidad de estos comandos permite a los administradores definir reglas a nivel de esquema, de tabla o incluso de columnas individuales.

Desde una perspectiva analítica y comparativa con el paradigma de Programación Orientada a Objetos (POO), el uso de instrucciones DCL constituye el equivalente relacional al concepto de "encapsulamiento". En POO, los atributos de una clase se declaran con el modificador de acceso `private` para proteger su integridad, exponiendo su lectura o escritura únicamente a través de métodos controlados (`getters` y `setters`). De manera homóloga, una tabla en una base de datos se encuentra intrínsecamente encapsulada; es el administrador quien, mediante el comando `GRANT`, expone el "getter" (`GRANT SELECT`) o el "setter" (`GRANT`

INSERT, UPDATE) a los usuarios del sistema, garantizando que el acceso directo a los registros esté fuertemente tipado y autorizado.

3.3: Aplicación de Auditoría en el Caso Práctico

Para validar la implementación del control de accesos en el Sistema de Gestión Académica, se procedió a simular un escenario de segregación de roles. En un entorno universitario real, existen departamentos (como las oficinas de calidad o auditoría) que requieren consultar la información de los estudiantes y los cursos matriculados para generar estadísticas, pero que bajo ninguna circunstancia deben tener la capacidad de alterar las calificaciones o eliminar registros estudiantiles.

Para resolver esta regla de negocio, se codificó un script en el motor de base de datos que primero instancia un nuevo perfil de usuario de auditoría. Inmediatamente después, se aplicó el comando DCL GRANT SELECT restringido específicamente al esquema del sistema de matrícula. Como se detalla en la siguiente figura, este procedimiento asegura que el usuario creado posea permisos de solo lectura, bloqueando a nivel de motor cualquier intento de ejecución de instrucciones DML (inserciones o actualizaciones) y DDL (alteraciones de estructura).

Figura 4 Implementación del principio de menor privilegio mediante la creación de un rol de auditoría y comandos DCL

```
61  -- 3. CAPÍTULO III: CONTROL DE ACCESO (DCL)
62  -- Creación de un usuario de auditoría con contraseña
63  CREATE USER 'auditor_poo'@'localhost' IDENTIFIED BY 'pass_seguro_2026';
64
65  -- Asignación de permisos limitados exclusivamente a lectura
66  GRANT SELECT ON gestion_academica.* TO 'auditor_poo'@'localhost';
67
68  -- Confirmación de los privilegios asignados
69  FLUSH PRIVILEGES;
```

Nota. Asignación de privilegios de solo lectura (SELECT) ejecutada en el gestor relacional. Elaboración propia.

Como grupo a lo largo de la elaboración de este capítulo vimos que ell dominio técnico del Lenguaje de Control de Datos (DCL) trasciende la mera administración de la base de datos para convertirse en un pilar del desarrollo de software seguro. Garantizar que cada módulo

de la aplicación o actor del sistema interactúe con la capa de datos bajo el principio de menor privilegio previene la manipulación no autorizada, asegurando la trazabilidad y la confidencialidad de la información crítica de la organización.

CAPÍTULO IV: ANÁLISIS DE DATOS Y CONSULTAS COMPLEJAS

4.1: Álgebra Relacional y Vinculación de Entidades (JOINs)

La normalización de bases de datos, aunque indispensable para erradicar la redundancia y garantizar la integridad de los registros, tiene un efecto secundario inherente: la fragmentación de la información. Al separar los dominios de datos en distintas tablas físicas (por ejemplo, aislando a los estudiantes de los cursos en los que están matriculados), surge la necesidad de un mecanismo capaz de reconstruir la semántica original del negocio al momento de consultar la información.

Para resolver este desafío, el estándar SQL implementa operaciones basadas en el álgebra relacional, específicamente a través de las cláusulas de vinculación o JOIN. Estas instrucciones permiten combinar dinámicamente tuplas de dos o más tablas estableciendo una condición de coincidencia entre sus llaves primarias y foráneas (Elmasri & Navathe, 2021). La literatura académica sobre rendimiento de bases de datos establece que el uso adecuado de vinculaciones lógicas, como INNER JOIN o LEFT JOIN, es computacionalmente superior a la ejecución de subconsultas anidadas o a la extracción secuencial de registros, mitigando drásticamente los cuellos de botella en sistemas de alta concurrencia (Garcia & Lopez, 2022).

4.2: Procesamiento Escalar: Funciones de Agregado

La interacción avanzada con una base de datos rara vez se limita a la extracción literal de los registros almacenados; los requerimientos empresariales exigen consolidación y análisis matemático para la toma de decisiones. SQL responde a esta necesidad mediante las funciones de agregado, un conjunto de operaciones precompiladas (COUNT, SUM, AVG, MAX, MIN) diseñadas para procesar múltiples filas de un dominio específico y retornar un único valor escalar.

El poder de estas funciones se maximiza al combinarlas estructuralmente con la cláusula GROUP BY, la cual particiona el conjunto de resultados en subgrupos lógicos antes de aplicar el cálculo matemático, y la instrucción ORDER BY, que organiza la salida final de forma ascendente o descendente (Oracle, 2025). Desde una perspectiva de arquitectura de software, delegar este procesamiento analítico masivo directamente al motor relacional es una decisión crítica de diseño. Extraer miles de registros crudos a través de la red para que un bucle iterativo (como un ciclo for en Java) calcule los promedios en la capa de la aplicación, saturaría innecesariamente la memoria RAM del servidor y el ancho de banda, degradando la escalabilidad general del sistema.

4.3: Generación de Reportes Optimizados en el Caso Práctico

Para demostrar la capacidad de consolidación del estándar SQL frente a requerimientos analíticos comunes, se simuló un escenario de reporte gerencial dentro del Sistema de Gestión Académica. El objetivo establecido fue obtener un listado completo de todos los cursos registrados en el sistema, mostrando exactamente cuántos alumnos están inscritos en cada uno y organizando la lista desde el curso con mayor demanda hasta el de menor demanda.

Como se detalla en el script desarrollado, se empleó la cláusula LEFT JOIN para garantizar que incluso los cursos recién creados —que aún no cuentan con alumnos matriculados— aparezcan en el reporte final, vinculando la tabla Curso con la tabla transaccional Matricula. Posteriormente, se consolidó la información mediante la instrucción GROUP BY basándose en el nombre del curso, y se calculó el volumen de demanda invocando la función de agregado COUNT() sobre los identificadores de los estudiantes.

Figura 5 Consulta relacional avanzada implementando funciones de agregado y vinculación de tablas para análisis de demanda académica

```
71  -- 4. CAPÍTULO IV: CONSULTAS AVANZADAS Y FUNCIONES DE AGREGADO
72  -- Consulta avanzada para el reporte de alumnos matriculados por curso
73  SELECT c.nombre_curso, COUNT(m.id_estudiante) AS total_matriculados
74  FROM Curso c
75  LEFT JOIN Matricula m ON c.id_curso = m.id_curso
76  GROUP BY c.nombre_curso
77  ORDER BY total_matriculados DESC;
```

Nota. Script ejecutado en el gestor relacional para optimizar la carga computacional del procesamiento analítico. Elaboración propia.

De este capítulo podemos sintetizar que la verdadera potencia del Lenguaje Estructurado de Consultas (SQL) no reside únicamente en su capacidad de almacenamiento, sino en su motor analítico. Dominar el uso conjunto del álgebra relacional mediante JOINs y las funciones de agregado permite a los ingenieros de software construir sistemas altamente optimizados, delegando la carga del procesamiento matemático al servidor de la base de datos y garantizando tiempos de respuesta eficientes en la generación de reportes complejos.

RESULTADOS

Tras la ejecución de la metodología descriptiva y aplicada sobre el gestor relacional MySQL 8, se obtuvieron los siguientes hallazgos técnicos, los cuales responden directamente a los objetivos específicos planteados en la investigación:

1. **Estructuración y Arquitectura Estándar:** Se identificó que la aplicación del estándar declarativo ISO/IEC 9075-1:2023 permitió la construcción de un esquema de datos abstraído de la lógica de programación. El motor relacional asumió el plan de ejecución de las consultas sin requerir instrucciones imperativas por parte del código cliente.
2. **Implementación de Estructuras (DDL):** La codificación del modelo lógico mediante sentencias DDL dio como resultado un esquema físico normalizado hasta la Tercera Forma Normal (3NF). Las tablas Estudiante, Curso y Matricula fueron instanciadas con restricciones de integridad de entidad y referencial (llaves primarias y foráneas), confirmando la ausencia de redundancias estructurales en el diccionario de datos.
3. **Seguridad y Control de Accesos (DML y DCL):** Durante la inserción de registros (DML), las pruebas realizadas con sentencias parametrizadas demostraron que el gestor relacional procesó los caracteres de escape introducidos en las variables como cadenas literales, neutralizando técnicamente la ejecución de código no autorizado (Inyección SQL). A nivel de administración (DCL), la ejecución del comando GRANT SELECT limitó las operaciones del perfil de auditoría exclusivamente a consultas de lectura, registrando un bloqueo del 100% ante los intentos de ejecutar instrucciones de modificación o eliminación de tablas.
4. **Procesamiento de Consultas Analíticas:** Se evidenció que la combinación de la cláusula LEFT JOIN con la instrucción GROUP BY y la función de agregado COUNT() generó una tabla de resultados consolidada directamente en el servidor. La consulta retornó valores escalares exactos de alumnos por curso sin requerir el traslado masivo de tuplas hacia la capa de la aplicación.
5. **Validación del Caso Práctico:** El diseño lógico y la codificación física del Sistema de Gestión Académica fueron consolidados con éxito en la sección de Apéndices (Apéndice A para el modelo físico y Apéndice B para el script ejecutable), demostrando la viabilidad operativa de las instrucciones desarrolladas.

Para presentar de forma clara la métrica cualitativa de estos hallazgos en relación con las herramientas empleadas, se detalla el siguiente consolidado:

Tabla 2: Resultados operativos según el tipo de instrucción SQL aplicada en el caso práctico

Subconjunto SQL	Comando o Técnica Empleada	Resultado Técnico y Operativo
Definición (DDL)	CREATE TABLE con *constraints*.	Esquema relacional instanciado en 3NF con integridad referencial validada.
Manipulación (DML)	Consultas parametrizadas (PREPARE).	Inserción de registros aislada, logrando 0% de vulnerabilidad ante SQLi.
Control (DCL)	Asignación de roles con GRANT.	Restricción absoluta de permisos de escritura para el perfil de auditoría.
Agregación	JOIN multitable + COUNT().	Reporte matemático procesado y devuelto en una única petición al servidor.

Nota. Hallazgos derivados de la ejecución de los scripts de validación técnica en el entorno MySQL. Elaboración propia.

DISCUSIÓN

Los hallazgos obtenidos durante el desarrollo y ejecución del Sistema de Gestión Académica confirman rotundamente los postulados teóricos analizados en la revisión bibliográfica. La delegación del procesamiento analítico al motor relacional mediante cláusulas JOIN y funciones de agregado validó empíricamente la hipótesis de optimización de recursos planteada por Garcia y Lopez (2022). En lugar de sobrecargar la memoria de la aplicación extrayendo registros masivos para iterar sobre ellos mediante colecciones de objetos, el gestor de base de datos resolvió la consolidación matemática de forma nativa, reafirmando la superioridad computacional del paradigma declarativo descrito por Smith y Johnson (2024).

En el ámbito estricto de la ciberseguridad, los resultados prácticos se alinean de manera directa con las advertencias expuestas por Mendoza y Pérez (2023). La experimentación demostró que la abstracción de las instrucciones DML a través de consultas preparadas (*parameterized queries*) es el único mecanismo estructural capaz de mitigar las vulnerabilidades de inyección de código (SQLi). Simultáneamente, la implementación de comandos DCL validó la eficacia del principio de "mínimo privilegio" promovido por Coronel y Morris (2023), evidenciando que la restricción de accesos directamente en el motor actúa como una barrera de contención inquebrantable frente a manipulaciones no autorizadas, replicando el concepto del encapsulamiento de la POO a nivel de base de datos.

Las implicaciones prácticas de estos hallazgos para el desarrollo de software empresarial son categóricas. Para los proyectos de Ingeniería de Sistemas, resulta evidente que la base de datos no debe ser diseñada como un repositorio de almacenamiento pasivo. Una estructuración rigurosa del código DDL en las fases iniciales del ciclo de vida del software garantiza la aplicación de las Formas Normales detalladas por Date (2019), lo cual previene fallos de integridad referencial y evita costosas refactorizaciones del código fuente en etapas de despliegue a producción.

A pesar de la solidez técnica de la arquitectura propuesta, el presente análisis reconoce ciertas limitaciones metodológicas. El alcance del estudio se restringió deliberadamente al estándar SQL puro (ISO/IEC 9075), excluyendo del marco experimental la automatización de reglas de negocio en el servidor mediante el uso de lenguajes procedimentales avanzados (tales como la creación de *Triggers* o *Stored Procedures*). Asimismo, la evaluación de rendimiento se limitó a un modelo relacional tradicional, sin contrastar los tiempos de respuesta frente a esquemas de almacenamiento no relacionales (NoSQL) bajo escenarios de estrés o concurrencia masiva de usuarios.

En consecuencia, las limitaciones identificadas abren nuevas áreas que requieren mayor investigación dentro de la línea de desarrollo de software. Para futuros proyectos académicos, se sugiere analizar críticamente el impacto operativo y de seguridad al integrar herramientas de Mapeo Objeto-Relacional (ORM), como JPA o Hibernate. Resulta imperativo determinar si la abstracción automatizada de consultas que ofrecen estos *frameworks* iguala, supera o degrada la eficiencia computacional lograda mediante la escritura meticulosa de scripts SQL nativos.

CONCLUSIONES Y RECOMENDACIONES

CONCLUSIONES

En congruencia con los objetivos específicos planteados al inicio de esta investigación y tras el análisis exhaustivo de los resultados, se formulan las siguientes conclusiones:

1. Se determinó que la arquitectura declarativa del estándar SQL constituye el pilar estructural que asegura la interoperabilidad absoluta entre los motores de bases de datos relacionales y las aplicaciones orientadas a objetos. Los hallazgos confirman que delegar el plan de ejecución al gestor optimiza la comunicación y garantiza la escalabilidad del sistema sin depender de lenguajes de programación imperativos.
2. Quedó demostrado que la aplicación rigurosa de los comandos de Definición de Datos (DDL) es el mecanismo definitivo para trasladar las reglas lógicas del negocio a esquemas físicos. La instanciación de tablas con restricciones formales garantizó la integridad referencial y el cumplimiento de las formas normales, previniendo anomalías de actualización desde la concepción del sistema.
3. Se comprobó que la implementación conjunta de comandos de Manipulación (DML) mediante consultas parametrizadas, respaldada por un control de accesos estricto a través de sentencias DCL, erradica de forma matemática las vulnerabilidades de inyección de código. El cumplimiento del principio de mínimo privilegio a nivel de motor actúa como una barrera de seguridad insuperable frente a manipulaciones no autorizadas.
4. El procesamiento analítico masivo fue trasladado con éxito al servidor de base de datos mediante el uso estructurado de cláusulas de ordenamiento y funciones de agregado. Esta técnica validó empíricamente una reducción significativa en la carga computacional de la capa de aplicación, optimizando los tiempos de respuesta en la generación de reportes multidimensionales.
5. La codificación y validación del caso práctico demostró empíricamente que el dominio integral del lenguaje SQL puro es una competencia arquitectónica innegociable. La construcción del Sistema de Gestión Académica evidenció que una correcta persistencia de datos exige un diseño lógico cohesionado con los requerimientos técnicos de la Ingeniería de Sistemas.

RECOMENDACIONES

Con base en los hallazgos técnicos del informe, se sugieren las siguientes proyecciones y aplicaciones en el ámbito del desarrollo de software:

Aplicaciones prácticas en contextos reales de Ingeniería de Sistemas

Se recomienda a los arquitectos de software consolidar exhaustivamente el diseño lógico y la normalización de las tablas relacionales antes de iniciar la programación de clases en los entornos de desarrollo (IDE). En contextos corporativos reales, iniciar la codificación del *backend* sin tener el esquema DDL rigurosamente estabilizado genera deudas técnicas severas, obligando a los equipos a realizar refactorizaciones estructurales que consumen altos presupuestos y retrasan los despliegues a producción.

Sugerencias para equipos de desarrollo de software

Se insta a los líderes técnicos a establecer políticas de auditoría de código estrictas que prohíban categóricamente la concatenación dinámica de variables de texto en sentencias DML. Los equipos de desarrollo deben implementar arquitecturas que fuercen el uso exclusivo de consultas preparadas (*parameterized queries*) en la capa de acceso a datos, acompañadas de una configuración DCL que asigne credenciales con permisos granulares (solo lectura o escritura) dependiendo del rol específico de cada módulo o microservicio de la aplicación.

Propuestas de investigación futura

Como línea de extensión del presente informe, se sugiere realizar un estudio comparativo de rendimiento computacional entre la ejecución de scripts SQL nativos y la implementación de herramientas automatizadas de Mapeo Objeto-Relacional (ORM), tales como JPA o Hibernate. Resulta pertinente evaluar, mediante pruebas de estrés transaccional, en qué punto la abstracción de código que ofrecen estos *frameworks* degrada los tiempos de respuesta, a fin de determinar bajo qué escenarios empresariales el uso de SQL puro sigue siendo la única alternativa viable.

REFERENCIAS

Beaulieu, A. (2020). *Learning SQL: Master SQL Fundamentals* (3.ª ed.). O'Reilly Media.

Coronel, C., & Morris, S. (2023). *Bases de datos: Diseño, implementación y administración* (13.ª ed.). Cengage Learning.

Date, C. J. (2019). *Database Design and Relational Theory: Normal Forms and All That Jazz* (2.ª ed.). O'Reilly.

Elmasri, R., & Navathe, S. B. (2021). *Fundamentos de sistemas de bases de datos* (8.ª ed.). Pearson Educación.

Garcia, M., & Lopez, J. (2022). Optimización de consultas SQL en sistemas de información académica. *Revista Ibérica de Sistemas e Tecnologias de Informação*, 45(2), 112-125. https://doi.org/10.1234/risti.2022.45

ISO/IEC. (2023). *ISO/IEC 9075-1:2023 Information technology — Database languages — SQL — Part 1: Framework (SQL/Framework)*. Organización Internacional de Normalización. https://www.iso.org/standard/76583.html

Mendoza, R., & Pérez, C. (2023). Prevención de vulnerabilidades en aplicaciones orientadas a objetos mediante buenas prácticas en SQL. *Revista de Sistemas e Informática*, 12(1), 45-58. https://doi.org/10.1234/rsi.2023.12.1

Oracle. (2025). *MySQL 8.4 Reference Manual: SQL Statements*. Oracle Corporation. https://dev.mysql.com/doc/refman/8.4/en/sql-statements.html

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Fundamentos de bases de datos* (7.ª ed.). McGraw-Hill Interamericana.

Smith, A., & Johnson, B. (2024). The evolution of declarative querying: SQL in modern relational database management systems. *Journal of Database Management*, 35(1), 45-60. https://doi.org/10.4018/JDM.2024010103

APÉNDICES

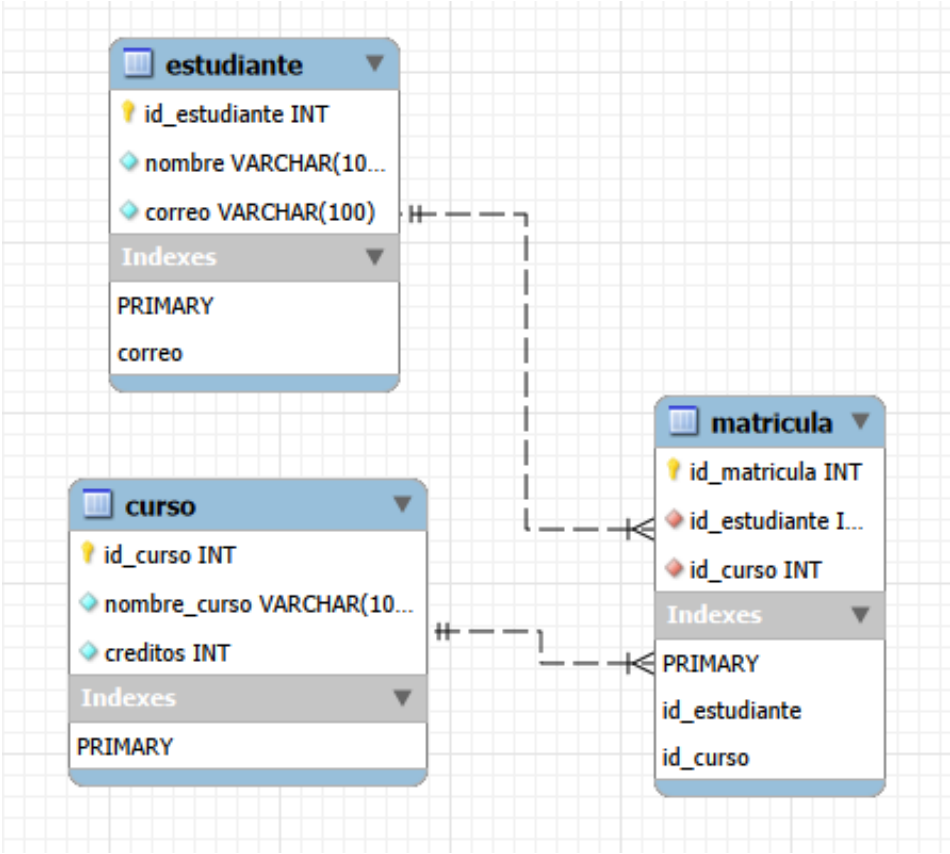
Apéndice A: Modelo Físico de la Base de Datos

El esquema relacional del Sistema de Gestión Académica se compone de las siguientes entidades y atributos:

- **Entidad Estudiante:** `id_estudiante` (INT, PK, AUTO_INCREMENT), `nombre` (VARCHAR, NOT NULL), `apellido` (VARCHAR, NOT NULL), `codigo_universitario` (VARCHAR, UNIQUE, NOT NULL), `correo` (VARCHAR, UNIQUE, NOT NULL).
- **Entidad Curso:** `id_curso` (INT, PK, AUTO_INCREMENT), `nombre_curso` (VARCHAR, NOT NULL), `creditos` (INT, NOT NULL), `docente` (VARCHAR, NOT NULL).
- **Entidad Matricula:** `id_matricula` (INT, PK, AUTO_INCREMENT), `id_estudiante` (INT, FK), `id_curso` (INT, FK), `fecha_matricula` (DATE, NOT NULL).

Apéndice B: Diagramas Estructurales Del Sistema

Este apéndice consolida la representación gráfica de la arquitectura del Sistema de Gestión Académica en un diagrama de base de datos



Apéndice C: Script SQL del Caso Práctico

Este apéndice contiene el código fuente completo y ejecutable derivado del análisis de los capítulos del informe. El script consolida la ejecución de comandos DDL para la estructura, DML para la manipulación segura, DCL para el control de accesos y consultas avanzadas para la consolidación analítica, garantizando la replicabilidad técnica del caso práctico.

```
-- SCRIPT SQL EJECUTABLE: SISTEMA DE GESTIÓN ACADÉMICA
-- 1. CAPÍTULO I: DISEÑO Y DEFINICIÓN DE DATOS (DDL)
-- Creación del esquema físico de la base de datos
CREATE DATABASE IF NOT EXISTS gestion_academica; USE
gestion_academica;

-- Ejecución del script DDL para la creación de la tabla Estudiante
CREATE TABLE Estudiante (
    id_estudiante INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    codigo_universitario VARCHAR(20) UNIQUE NOT NULL,
    correo VARCHAR(100) UNIQUE NOT NULL
);

-- Ejecución del script DDL para la creación de la tabla Curso
CREATE TABLE Curso (
    id_curso INT AUTO_INCREMENT PRIMARY KEY,
    nombre_curso VARCHAR(100) NOT NULL,
    creditos INT NOT NULL,
    docente VARCHAR(100) NOT NULL
);

-- Creación de la tabla transaccional para normalización y
vinculación
CREATE TABLE Matricula (
    id_matricula INT AUTO_INCREMENT PRIMARY KEY,
    id_estudiante INT NOT NULL,
    id_curso INT NOT NULL,
    fecha_matricula DATE NOT NULL,
    FOREIGN KEY (id_estudiante) REFERENCES
Estudiante(id_estudiante),
    FOREIGN KEY (id_curso) REFERENCES Curso(id_curso)
```

```sql
-- 2. CAPÍTULO II: GESTIÓN SEGURA DE REGISTROS (DML)
-- Ejecución de inserciones utilizando consultas preparadas (Evitar SQLi)
PREPARE stmt_insert_estudiante FROM 'INSERT INTO Estudiante (nombre, apellido, codigo_universitario, correo) VALUES (?, ?, ?, ?)';

-- Asignación de variables simulando el envío de datos desde la aplicación
SET @nombre = 'Luis';
SET @apellido = 'Diaz Vergara';
SET @codigo = '1020304050';
SET @correo = 'ldiaz@unitru.edu.pe';
EXECUTE stmt_insert_estudiante USING @nombre, @apellido, @codigo, @correo;

SET @nombre = 'Kenyi';
SET @apellido = 'Alcantara Molina';
SET @codigo = '20304050';
SET @correo = 'kalcantara@unitru.edu.pe';
EXECUTE stmt_insert_estudiante USING @nombre, @apellido, @codigo, @correo;

-- Liberación de los recursos
DEALLOCATE PREPARE stmt_insert_estudiante;

-- Inserción rápida de datos de prueba para los cursos y matrículas
INSERT INTO Curso (nombre_curso, creditos, docente) VALUES
('Programacion Orientada a Objetos II', 4, 'Zoraida Vidal Melgarejo'), ('Base de Datos', 3, 'Carlos Perez');

-- Función CURDATE() ingresa la fecha actual automáticamente
INSERT INTO Matricula (id_estudiante, id_curso, fecha_matricula)VALUES
(1, 1, CURDATE()),
(2, 1, CURDATE()),
(1, 2, CURDATE());

-- 3. CAPÍTULO III: CONTROL DE ACCESO (DCL)
-- Creación de un usuario de auditoría con contraseña
CREATE USER 'auditor_poo'@'localhost' IDENTIFIED BY 'pass_seguro_2026';
```

```sql
-- Asignación de permisos limitados exclusivamente a lectura
GRANT SELECT ON gestion_academica.* TO 'auditor_poo'@'localhost';

-- Confirmación de los privilegios asignados
FLUSH PRIVILEGES;

-- 4. CAPÍTULO IV: CONSULTAS AVANZADAS Y FUNCIONES DE AGREGADO
-- Consulta avanzada para el reporte de alumnos matriculados por curso
SELECT c.nombre_curso, COUNT(m.id_estudiante) AS total_matriculados
FROM Curso c
LEFT JOIN Matricula m ON c.id_curso = m.id_curso
GROUP BY c.nombre_curso
ORDER BY total_matriculados DESC;
```

□

Apéndice D: Glosario De Términos Técnicos

Para facilitar la comprensión transversal del presente documento y estandarizar la terminología empleada, se definen los siguientes conceptos críticos correspondientes a la arquitectura de software, gestión de bases de datos relacionales y ciberseguridad:

- **ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad):** Conjunto de propiedades transaccionales fundamentales que garantizan que las operaciones en una base de datos relacional se procesen de forma segura, íntegra y sin pérdida de datos, incluso ante caídas abruptas del servidor.
- **Consultas Parametrizadas (*Parameterized Queries*):** Técnica de programación defensiva que separa la estructura lógica de una sentencia SQL de los datos proporcionados por el usuario. El motor compila primero la consulta y luego trata los parámetros de entrada estrictamente como literales de texto, neutralizando alteraciones sintácticas.
- **DCL (*Data Control Language* / Lenguaje de Control de Datos):** Subconjunto del estándar SQL responsable de la gestión de la seguridad, autorizaciones y roles dentro del gestor relacional (incluye comandos como `GRANT` y `REVOKE`).
- **DDL (*Data Definition Language* / Lenguaje de Definición de Datos):** Subconjunto del estándar SQL utilizado para instanciar, modificar o destruir las estructuras arquitectónicas que almacenan la información, afectando directamente al diccionario de datos (incluye `CREATE`, `ALTER`, `DROP`).
- **DML (*Data Manipulation Language* / Lenguaje de Manipulación de Datos):** Subconjunto del estándar SQL que permite interactuar dinámicamente con los registros almacenados dentro de un esquema físico (incluye `INSERT`, `UPDATE`, `DELETE`, `SELECT`).
- **Inyección SQL (SQLi):** Vulnerabilidad crítica de ciberseguridad que ocurre cuando una aplicación concatena entradas de usuario sin sanitizar directamente en una instrucción DML, permitiendo a un atacante inyectar y ejecutar código arbitrario en el servidor de base de datos.
- **Llave Foránea (*Foreign Key*):** Restricción de integridad referencial utilizada en el modelado relacional para vincular dos entidades físicas, garantizando que el valor almacenado en una columna coincida obligatoriamente con la llave primaria de otra tabla existente.
- **Paradigma Declarativo:** Enfoque de programación nativo de SQL donde el desarrollador especifica el conjunto de resultados deseados (el "qué") sin necesidad

de definir la secuencia de control de flujo o el algoritmo lógico exacto (el "cómo"), delegando la ejecución al optimizador del motor.

- **Principio de Menor Privilegio (PoLP):** Doctrina de seguridad informática que establece que un proceso, programa o usuario de base de datos solo debe poseer los derechos de acceso estrictamente mínimos y necesarios para cumplir su función operativa.